

VAIJUNTO · SISTEMA DE CARONAS COMPARTILHADAS

Cliente e Servidor TCP em Go

Um guia linha por linha para quem vem de Python e Java

Disciplina Concorrência e Conectividade (PBL)

Exemplo `Aprendizado/01-eco`

Versão do Go 1.27

Pré-requisito lógica de programação; nenhum conhecimento de redes

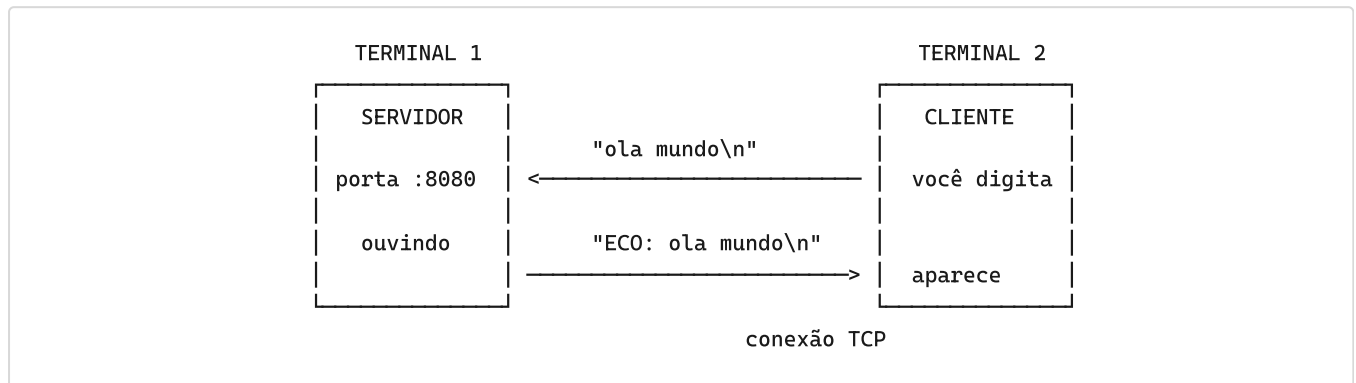
Sumário

1. O que vamos construir — e por que um programa bobo ensina tanto
2. Go visto de Python e de Java — o mapa mental
3. Anatomia de um programa Go — pacote, import, go.mod
4. As seis estranhezas — o que mais choca quem vem de fora
5. Como um socket TCP funciona — a teoria mínima
6. O servidor, linha por linha
7. O cliente, linha por linha
8. O problema que o TCP não resolve — e por que existe o bufio
9. Concorrência: o que já temos e o perigo que vem
10. Como rodar e três experimentos
11. Cola de tradução — Python, Java e Go lado a lado
12. Próximos passos rumo ao Vaijunto

1. O que vamos construir

Um **servidor de eco**. Você digita uma frase no cliente, ela viaja pela rede, o servidor recebe e devolve a mesma frase com `ECO:` na frente.

É deliberadamente bobo. O valor não está no que o programa faz, está em **toda a mecânica de rede e concorrência do projeto final já estar presente aqui dentro**, em menos de cem linhas. Trocar o texto `ECO:` por “reservar assento na carona 42” é o trabalho do resto do semestre; o esqueleto não muda.



O fluxo completo: dois programas independentes conversando por uma conexão TCP.

Os arquivos

Aprendizado/01-eco/	
go.mod	identidade do projeto
servidor/main.go	abre a porta e atende clientes
cliente/main.go	conecta e conversa

POR QUE DOIS PROGRAMAS SEPARADOS

Cliente e servidor são **executáveis distintos**, que rodam em processos distintos e poderiam estar em máquinas distintas. Isso não é detalhe de organização: é a essência da matéria. Eles não compartilham memória, não enxergam as variáveis um do outro. A **única** coisa que os liga é uma sequência de bytes atravessando a rede.

2. Go visto de Python e de Java

Antes de qualquer linha de código, vale posicionar a linguagem. Go foi criada no Google em 2009 justamente para escrever servidores, e cada decisão de projeto dela responde à pergunta “como fazer software de rede sem sofrer”.

Aspecto	Python	Java	Go
Tipagem	Dinâmica, erro só ao rodar	Estática, muito verbosa	Estática, mas inferida — parece Python, protege como Java
Execução	Interpretada	Bytecode na JVM	Compilada para binário nativo
Distribuição	Precisa do interpretador	Precisa da JVM	Um arquivo só , sem nada instalado
Erros	<code>try / except</code>	<code>try / catch</code>	Erro é valor de retorno , sem exceções
Concorrência	Threads travadas pelo GIL	Threads reais, mas caras	Goroutines : reais e baratíssimas
Herança	Sim, e múltipla	Sim	Não existe — só composição
Tamanho	35 palavras-chave	50+ palavras-chave	25 palavras-chave
Formatação	PEP8, opcional	Depende da IDE	<code>gofmt</code> oficial: acabou a discussão

A BOA NOTÍCIA

Go é menor que as duas linguagens que você já conhece. Não há herança, sobrecarga de operador, construtores, anotações, decoradores, generics no dia a dia, nem exceções. A curva não é longa — é curta e íngreme: poucas ideias novas, mas que precisam ser realmente entendidas.

A frase que resume a filosofia

Em Java você descreve *o que as coisas são* (classes, hierarquias, interfaces). Em Python você descreve *o que quer*, e a linguagem se vira. Em Go você descreve **exatamente o que a máquina faz, passo a passo, sem mágica escondida**. É mais explícito e às vezes mais repetitivo — e é precisamente por isso que serve para um trabalho cujo objetivo é você entender cada passo.

3. Anatomia de um programa Go

O arquivo go.mod

Aprendizado/01-eco/go.mod

```
module vaijunto/aprendizado/eco

go 1.27
```

É o `requirements.txt` do Python ou o `pom.xml` do Maven — só que quase vazio, porque **este projeto não tem nenhuma dependência externa**. Tudo que usamos vem com a linguagem.

- `module` dá um nome ao projeto. É o identificador usado se alguém importar seu código.
- `go 1.27` declara a versão mínima da linguagem.

Quando um dia você precisar de uma biblioteca de terceiros, o comando `go get` adiciona uma linha `require` aqui. Para este projeto, ele vai continuar assim.

O cabeçalho de todo arquivo .go

```
package main

import (
    "bufio"
    "fmt"
    "net"
)
```

`package main`

Todo arquivo Go pertence a um pacote. `main` é um nome mágico: significa “isto é um programa executável”. Qualquer outro nome produziria uma biblioteca, sem como rodar.

Equivalência mental: é o `if __name__ == "__main__"` do Python, ou a classe que tem o `public static void main` em Java — só que declarado no topo do arquivo, não escondido no meio dele.

`import`

Traz pacotes para o arquivo. Os três aqui vêm todos da biblioteca padrão:

- `net` — a rede. É onde vive o socket TCP.
- `bufio` — leitura com buffer. O motivo aparece na seção 8.
- `fmt` — formatação e impressão de texto. O `print` do Python, o `System.out` do Java.

ESTRANHEZA NÚMERO ZERO

Importar um pacote e não usar é erro de compilação. Não é aviso, não é lint: o programa simplesmente não compila. O mesmo vale para variáveis declaradas e não usadas.

Incomoda muito nos primeiros dias — em Python e Java isso passa batido para sempre. Mas o efeito é que **não existe código morto** em projeto Go, e é impossível deixar um import esquecido de uma tentativa abandonada.

```
func main()
```

A função `main` dentro do pacote `main` é o ponto de entrada. Quando o programa inicia, a execução começa nela; quando ela termina, **o processo inteiro morre**. Guarde essa última parte, ela volta a importar na seção 9.

4. As seis estranhezas

Estas são as ideias que fazem alguém vindo de Python ou Java travar. Vale ler esta seção com calma: depois dela, o código dos capítulos 6 e 7 fica quase óbvio.

4.1 O operador `:=`

```
ouvinte, err := net.Listen("tcp", ":8080") // cria e atribui
err = outraCoisa()                       // só atribui, já existe
```

`:=` declara a variável **e** atribui o valor, deduzindo o tipo sozinho. É a ponte entre os dois mundos que você conhece: você escreve com a leveza do Python, mas o tipo fica **fixo e verificado pelo compilador** como em Java.

Depois de criada, usa-se `=`. Confundir os dois é o erro mais comum da primeira semana. A regra: `:=` só na primeira vez.

4.2 Retorno múltiplo e erro como valor

```
conexao, err := ouvinte.Accept()
if err != nil {
    fmt.Println("deu errado:", err)
    return
}
// daqui para baixo, conexao é confiável
```

Esta é a maior diferença cultural. Em Python e em Java, uma função que falha **lança** uma exceção, e ela sobe pela pilha até alguém capturar. Em Go **não existem exceções**. Uma função que pode falhar devolve dois valores: o resultado e o erro, lado a lado.

`nil` é o “nada” de Go — o `None` do Python, o `null` do Java. Então `if err != nil` lê-se literalmente: “o erro é diferente de nada? então algo falhou”.

Java / Python	Go
O erro é invisível na assinatura da função	O erro está escrito no retorno; impossível ignorar sem querer
Pode ser tratado longe de onde nasceu	É tratado na linha seguinte , sempre
Um <code>catch</code> genérico esconde tudo	Cada falha é decidida individualmente
Código enxuto no caminho feliz	Repetitivo — a crítica justa que se faz a Go

VALE A PENA?

Em um servidor, sim. A pergunta “o que acontece se *esta* operação falhar?” tem resposta diferente em cada ponto: às vezes derruba tudo, às vezes ignora e segue, às vezes tenta de novo. Você vai ver essa decisão sendo tomada de forma diferente três vezes no nosso servidor. Com exceções, essas três situações pareceriam iguais.

4.3 defer

```
ouvinte, err := net.Listen("tcp", ":8080")
if err != nil { return }
defer ouvinte.Close() // roda quando a função acabar, aconteça o que acontecer
```

`defer` agenda uma linha para ser executada **quando a função terminar** — por retorno normal, por `return` no meio, ou por `panic`. É o `finally` do Java, o `with` do Python, o `try-with-resources`.

A vantagem sobre os três: você escreve o fechamento **uma linha depois** da abertura, e não vinte linhas abaixo, no fim de um bloco. Abriu, já garantiu que fecha. Isso praticamente elimina a classe de bugs de recurso vazado — que, em um servidor, significa acumular conexões mortas até travar a máquina.

4.4 Goroutines

```
go atender(conexao) // roda em paralelo; não espera terminar
```

A palavra `go` na frente de uma chamada faz a função rodar **em paralelo**, e a execução continua imediatamente na linha seguinte sem esperar.

Compare com o que você conhece:

Python	Java	Go
<pre>t = Thread(target=f) t.start()</pre>	<pre>new Thread(() -> f()) .start();</pre>	<pre>go f()</pre>
Trava no GIL: não roda em paralelo de verdade	Paralelismo real, mas ~1 MB por thread	Paralelismo real, ~2 KB por goroutine
Centenas, com esforço	Alguns milhares no limite	Centenas de milhares, tranquilamente

A diferença de custo não é um detalhe de performance: ela **muda a arquitetura possível**. Em Java, uma thread por cliente conectado é caro o bastante para você precisar de pools e filas. Em Go, uma goroutine por cliente é o jeito normal, óbvio e recomendado de escrever. O código fica linear e legível, e mesmo assim escala.

4.5 Valor ou ponteiro

```
func mudar(c Carona) { c.Assentos = 0 } // mexe numa CÓPIA, não muda nada
func mudar(c *Carona) { c.Assentos = 0 } // mexe no ORIGINAL
```

Em Python e em Java, objetos são passados por referência: se a função altera o objeto, a alteração aparece fora. **Em Go, structs são copiados por padrão**. Passar um `struct` para uma função entrega a ela um clone.

O sintoma clássico do iniciante: “eu alterei o assento e não mudou nada”. Alterou — na cópia, que morreu no fim da função.

REGRA PRÁTICA

Se a função só **lê**, passe o valor. Se ela precisa **modificar**, use `*`. Você vai usar isso o tempo todo no Vaijunto, porque reservar assento é modificar.

4.6 O descarte `_`

```
_, err = fmt.Fprint(conexao, texto) // ignoro o 1º retorno de propósito
```

Lembra que variável não usada é erro de compilação? O `_` é como você diz explicitamente ao compilador: “*sei que existe um valor aqui e estou descartando de propósito*”. Descarte acidental deixa de existir; ele passa a ser sempre uma decisão visível no código.

5. Como um socket TCP funciona

Esta é a teoria mínima. Sem ela o código da próxima seção vira decoreba.

O que é um socket

Um **socket** é a tomada por onde um programa fala com a rede. Uma vez conectado, você o trata como um cano de bytes: escreve de um lado, sai do outro. Todo o trabalho de partir os dados em pacotes, mandar pela placa de rede, remontar do outro lado e reordenar o que chegou fora de ordem é feito pelo sistema operacional.

O que o TCP garante

TCP garante	TCP não garante
Que os bytes chegam	Onde as mensagens começam e terminam
Que chegam na ordem enviada	Que chegam agrupados como você enviou
Que não vêm corrompidos	Que o outro lado entendeu o significado

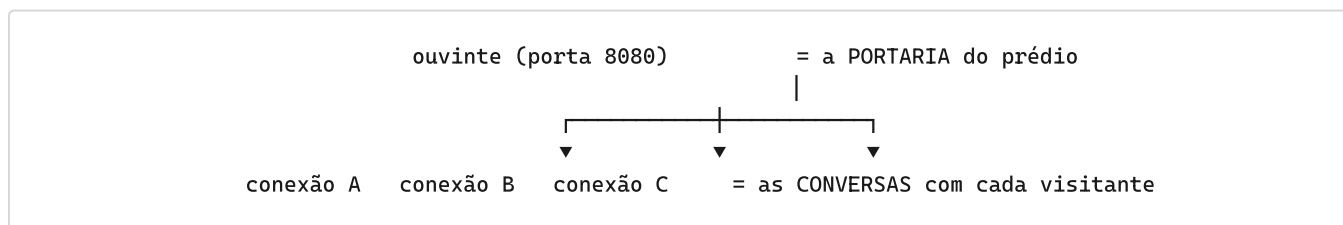
A coluna da direita é responsabilidade sua, e é o assunto da seção 8.

Os dois papéis

Cliente e servidor são assimétricos, e cada um tem seu próprio verbo:

Servidor — ESPERA	Cliente — PROCURA
<code>net.Listen("tcp", ":8080")</code> reserva a porta e fica de pé	<code>net.Dial("tcp", "host:8080")</code> vai até um endereço e conecta
<code>Accept()</code> — bloqueia até chegar alguém	Já sai com a conexão pronta
Precisa dizer em qual porta atende	Precisa dizer em quem quer falar

Ouvinte e conexão: a confusão clássica



Um ouvinte, muitas conexões. Cada `Accept()` produz uma conversa privada e independente.

O **ouvinte** não transporta dados. Ele só existe para produzir conexões, uma a cada `Accept()`. Cada **conexão** é um cano privado com um cliente específico e não sabe da existência das outras. Entender essa separação resolve metade das dúvidas de quem começa com sockets.

6. O servidor, linha por linha

Aprendizado/01-eco/servidor/main.go

```
package main

import (
    "bufio"
    "fmt"
    "net"
)

func main() {
    ouvinte, err := net.Listen("tcp", ":8080")
    if err != nil {
        fmt.Println("erro ao abrir a porta:", err)
        return
    }
    defer ouvinte.Close()

    fmt.Println("servidor ouvindo em :8080")

    for {
        conexao, err := ouvinte.Accept()
        if err != nil {
            fmt.Println("erro ao aceitar conexao:", err)
            continue
        }

        fmt.Println("cliente conectado:", conexao.RemoteAddr())
        go atender(conexao)
    }
}

func atender(conexao net.Conn) {
    defer conexao.Close()

    leitor := bufio.NewReader(conexao)

    for {
        mensagem, err := leitor.ReadString('\n')
        if err != nil {
            fmt.Println("cliente desconectou:", conexao.RemoteAddr())
            return
        }

        fmt.Printf("recebido de %v: %s", conexao.RemoteAddr(), mensagem)
        fmt.Fprintf(conexao, "ECO: %s", mensagem)
    }
}
```

Abrindo a porta

```
ouvinte, err := net.Listen("tcp", ":8080")
```

A linha mais importante do arquivo. Ela faz três coisas ao mesmo tempo: cria o socket, **reserva a porta 8080** junto ao sistema operacional, e começa a escutar.

- `"tcp"` — o protocolo. Poderia ser `"udp"`, mas o PBL exige TCP.
- `":8080"` — a porta. **Repare que não há nada antes dos dois-pontos.**

GUARDE ESTE DETALHE PARA O DOCKER

`":8080"` significa “*aceite conexões vindas de qualquer endereço de rede desta máquina*”. Se estivesse escrito `"localhost:8080"`, o servidor só aceitaria conexões da própria máquina — e **nada funcionaria entre contêineres**. É um erro clássico, difícil de diagnosticar, e você vai esbarrar nele na fase de Docker se não lembrar disso.

```
if err != nil {  
    fmt.Println("erro ao abrir a porta:", err)  
    return  
}
```

Na prática, esse erro quase sempre significa uma coisa: **a porta 8080 já está ocupada** — frequentemente por um servidor seu que ficou rodando esquecido. Aqui a decisão é `return`: sem porta não há servidor, então encerramos.

```
defer ouvinte.Close()
```

Devolve a porta ao sistema operacional quando `main` terminar. Escrito uma linha depois da abertura, como manda o costume.

O laço que aceita clientes

```
for {
```

`for` sem mais nada é o **laço infinito**. Go não tem `while`: o `for` faz todos os papéis. O servidor precisa de um laço infinito porque ele nunca “termina” — fica de pé esperando clientes até você matá-lo.

```
    conexao, err := ouvinte.Accept()
```

A segunda linha mais importante. `Accept()` **bloqueia**: a execução para ali, **sem consumir CPU**, até que alguém se conecte. Quando um cliente chega, devolve uma `conexao` exclusiva com ele. O `ouvinte` continua livre para receber os próximos.

```
if err != nil {
    fmt.Println("erro ao aceitar conexao:", err)
    continue
}
```

REPARE NA DIFERENÇA DE INTENÇÃO

Aqui usamos `continue`, e não `return` como no erro anterior. É uma decisão de projeto, não estilo: **um cliente com problema não pode derrubar o servidor inteiro**. Ignoramos aquela conexão e voltamos a atender os outros.

Este é exatamente o benefício de erro-como-valor da seção 4.2: dois erros, duas respostas diferentes, ambas visíveis no código. Com `try/catch` as duas situações pareceriam idênticas.

```
fmt.Println("cliente conectado:", conexao.RemoteAddr())
```

`RemoteAddr()` é o endereço de quem conectou. Ao testar, aparece algo como `:::1:57600` — o `:::1` é o `localhost` escrito em IPv6, e `57600` é a porta que o sistema sorteou para o cliente.

A linha que define o projeto

```
go atender(conexao)
```

Dispara `atender` como goroutine e volta **imediatamente** para `Accept()`, pronto para o próximo cliente.

Sem esse `go`, o servidor atenderia um cliente por vez, e o segundo ficaria na fila até o primeiro desconectar. Com ele, atende todos ao mesmo tempo. Uma palavra de duas letras é a diferença entre um brinquedo e um servidor.

Atendendo um cliente

```
func atender(conexao net.Conn) {
    defer conexao.Close()
}
```

Cada cliente conectado tem **sua própria execução desta função**, com sua própria variável `conexao`, isolada das demais. O `defer` garante que a conexão será fechada quando essa goroutine acabar, por qualquer motivo — sem isso o programa acumula conexões mortas até travar.

```
leitor := bufio.NewReader(conexao)

for {
    mensagem, err := leitor.ReadString('\n')
```

`ReadString('\n')` diz: **“me devolva bytes até encontrar uma quebra de linha”**. Ele espera, junta os pedaços que chegarem, e só retorna com a mensagem completa. O porquê disso ser necessário é o assunto da próxima seção.

Nota de sintaxe: `'\n'` com **aspas simples** é um caractere único; `"texto"` com aspas duplas é uma cadeia. Em Go são **tipos diferentes**, não uma escolha estética como em Python ou JavaScript.

E este `for` interno faz o cliente poder mandar quantas mensagens quiser na mesma conexão, sem reconectar.

```
if err != nil {
    fmt.Println("cliente desconectou:", conexao.RemoteAddr())
    return
}
```

Quando o cliente fecha a conexão, a leitura falha com um erro chamado **EOF** (fim de arquivo). No mundo dos sockets, **desconexão chega como erro de leitura** — é o comportamento normal e esperado, não uma falha.

O `return` encerra a função, o `defer` dispara e fecha a conexão. **Só esta goroutine morre**; o servidor e os demais clientes seguem intactos.

```
fmt.Printf("recebido de %v: %s", conexao.RemoteAddr(), mensagem)
```

`Printf` é o print formatado — o `%` do Python, o `String.format` do Java. Os marcadores são preenchidos na ordem: `%v` recebe o endereço, `%s` recebe a mensagem.

Detalhe fácil de não notar: **não há `\n` no fim dessa string**. Não precisa, porque `ReadString` **inclui o delimitador** no resultado — a quebra de linha já veio grudada na mensagem.

```
fmt.Fprintf(conexao, "ECO: %s", mensagem)
```

E aqui a resposta viaja de volta pela rede. `Fprintf` é o `Printf` com um **destino** no primeiro argumento (o `F` é de *file*).

A IDEIA MAIS ELEGANTE DO GO

Uma conexão de rede, um arquivo em disco e a tela do terminal são **a mesma coisa** para quem escreve: todos aceitam bytes. Trocar o destino é trocar um argumento — nada mais.

Consequência prática para o Vaijunto: quando você for **salvar as caronas em JSON**, vai usar exatamente estas mesmas funções, mudando apenas o destino de `conexao` para um arquivo.

7. O cliente, linha por linha

Aprendizado/01-eco/cliente/main.go

```
package main

import (
    "bufio"
    "fmt"
    "net"
    "os"
)

func main() {
    conexao, err := net.Dial("tcp", "localhost:8080")
    if err != nil {
        fmt.Println("nao consegui conectar:", err)
        return
    }
    defer conexao.Close()

    fmt.Println("conectado. digite uma mensagem e tecle enter (ctrl+c para sair)")

    teclado := bufio.NewReader(os.Stdin)
    servidor := bufio.NewReader(conexao)

    for {
        fmt.Print("> ")

        texto, err := teclado.ReadString('\n')
        if err != nil {
            return
        }

        _, err = fmt.Fprint(conexao, texto)
        if err != nil {
            fmt.Println("conexao perdida:", err)
            return
        }

        resposta, err := servidor.ReadString('\n')
        if err != nil {
            fmt.Println("servidor fechou a conexao")
            return
        }

        fmt.Print(resposta)
    }
}
```

Conectando

```
conexao, err := net.Dial("tcp", "localhost:8080")
```

O oposto de `Listen`. **O servidor espera; o cliente procura.** O cliente não abre porta nenhuma — vai até um endereço e bate na porta.

Note que aqui o endereço é **completo**: `"localhost:8080"`. Faz sentido — para conectar você precisa dizer *em quem*; o servidor só precisava dizer *em qual porta* atende.

A ÚNICA LINHA QUE MUDA NO DOCKER

`localhost` significa “esta mesma máquina”. Quando cliente e servidor forem para contêineres separados, essa string vira o nome do serviço do servidor — algo como `"servidor:8080"`. É praticamente a única alteração necessária no código.

O erro aqui quase sempre significa **servidor não está rodando**, ou endereço errado. Vale testar: rode o cliente sem o servidor e veja a mensagem aparecer.

Dois leitores, uma ideia bonita

```
teclado := bufio.NewReader(os.Stdin)
servidor := bufio.NewReader(conexao)
```

`os.Stdin` é a entrada padrão — o teclado. E repare: **o mesmo** `bufio.NewReader` **serve para os dois**.

Para o `bufio`, teclado e socket de rede são a mesma coisa: uma fonte de bytes. Ler uma linha que você digitou e ler uma linha que atravessou a internet usam **código idêntico**. É o espelho exato da ideia da caixa da página anterior, agora do lado da leitura.

Nomear as variáveis `teclado` e `servidor` faz o resto do código quase se explicar sozinho: lê do `teclado`, escreve na `conexao`, lê do `servidor`.

O ciclo de conversa

```
fmt.Print("> ")
```

`Print`, sem `ln` e sem `f`: imprime e **não quebra linha**. Por isso o cursor fica esperando ao lado do `>`, como um prompt de verdade.

```
texto, err := teclado.ReadString('\n')
if err != nil {
    return
}
```

Bloqueia até você digitar algo e apertar Enter — o `'\n'` aqui é literalmente a tecla Enter. Erro nesse ponto significa fim da entrada: `Ctrl+Z` e Enter no Windows, `Ctrl+D` no Linux.

```
_, err = fmt.Fprint(conexao, texto)
```

Manda o texto pela rede. Duas coisas para observar, ambas já vistas na seção 4:

- O `_` descarta o número de bytes escritos, que não interessa.
- É `=` e não `:=` porque `err` **já existe** desde a linha anterior.

```
resposta, err := servidor.ReadString('\n')
if err != nil {
    fmt.Println("servidor fechou a conexao")
    return
}

fmt.Print(resposta)
```

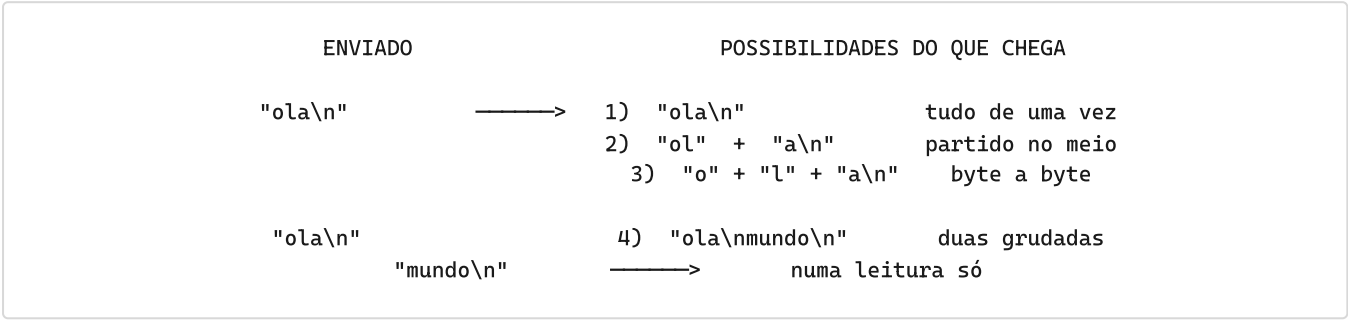
Espera a resposta chegar e imprime. `Print` sem quebra de linha de novo, porque a resposta já traz o `\n` que veio junto do texto original. E o laço recomeça.

8. O problema que o TCP não resolve

Aqui está o conceito que separa quem “fez funcionar” de quem **entendeu**. É também a fonte de bugs mais traiçoeira de programação em rede.

TCP é uma mangueira, não um carteiro

TCP não tem noção de “mensagem”. Ele entrega um fluxo contínuo de bytes. Se o cliente enviar `"ola\n"`, o servidor pode receber:



Os bytes chegam na ordem certa e sem perda. Onde eles são cortados é imprevisível.

As quatro situações são normais e legítimas. O que corta os dados são o tamanho dos pacotes de rede, o buffer do sistema operacional e o tráfego do momento — nada que você controle.

O BUG QUE ISSO CAUSA

Se você ler “o que vier” e tratar cada leitura como uma mensagem, seu programa **funciona perfeitamente no seu computador e quebra na apresentação**. Localmente as mensagens são pequenas e rápidas, e quase sempre caem no caso 1. Com dois contêineres, rede real e vários clientes, os casos 2 a 4 começam a acontecer — e aí meia mensagem vira um JSON inválido.

A solução: um protocolo de aplicação

A saída é **combinar uma regra** entre cliente e servidor sobre onde uma mensagem termina. Essa combinação tem nome: **protocolo de aplicação**. Existem três abordagens usuais:

Estratégia	Como funciona	Exemplos reais
Delimitador	Um caractere marca o fim — no nosso caso, <code>\n</code>	HTTP, SMTP, Redis, e o nosso eco
Tamanho no cabeçalho	Envia primeiro quantos bytes vêm, depois os bytes	gRPC, WebSocket
Tamanho fixo	Toda mensagem tem exatamente N bytes	Protocolos de hardware

Escolhemos a primeira, por ser a mais simples de depurar: você consegue ler o tráfego com os próprios olhos.

É isso que o bufio faz

```
leitor := bufio.NewReader(conexao)
mensagem, err := leitor.ReadString('\n')
```

O `bufio.Reader` envolve a conexão e acumula bytes num buffer interno. Aí você lê por **critério** em vez de ler pedaços brutos: ele guarda o que sobrou, espera o que falta, e devolve só quando a mensagem estiver completa. Os quatro casos do diagrama acima passam a ser tratados corretamente, sem você escrever uma linha para isso.

PARA O VAIJUNTO

Você vai manter exatamente esta estratégia, trocando o conteúdo da linha por JSON:

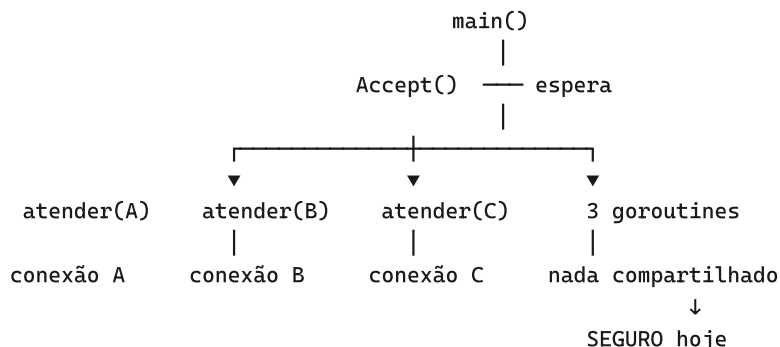
```
{"acao": "buscar", "origem": "Feira de Santana", "destino": "Salvador"}
{"acao": "reservar", "trechos": [7, 12]}
```

Uma linha, um comando. Simples de ler, simples de testar, simples de mostrar funcionando na apresentação.

9. Concorrência: o que temos e o perigo que vem

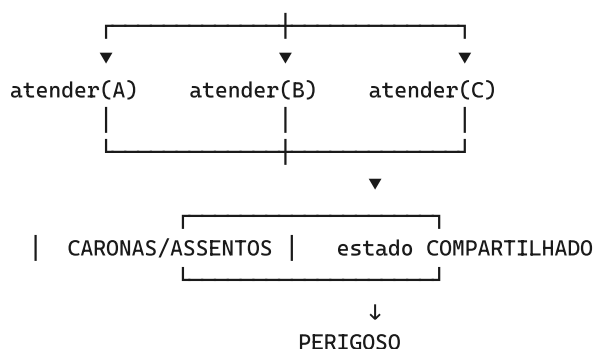
O que já funciona

Com `go atender(conexao)`, cada cliente ganha sua própria linha de execução. O teste com dois clientes simultâneos confirmou: ambos foram atendidos ao mesmo tempo, sem fila.



Neste exemplo **cada goroutine é uma ilha**. Ela só toca na própria conexão e não compartilha nada com ninguém. Por isso não há como dar errado — ainda.

O que vai acontecer no Vaijunto



No momento em que as goroutines passarem a mexer **no mesmo dado** — a lista de assentos livres — nasce a **condição de corrida**. Ela funciona assim:

Instante	Goroutine A (passageiro Ana)	Goroutine B (passageiro Bruno)
t1	lê: <code>assentos = 1</code>	—
t2	—	lê: <code>assentos = 1</code>
t3	confere: $1 > 0$, pode vender	confere: $1 > 0$, pode vender
t4	escreve: <code>assentos = 0</code>	escreve: <code>assentos = 0</code>
Resultado: duas passagens vendidas para o mesmo assento, e o contador marca 0.		

Nada “deu erro”. Nenhuma exceção, nenhum log. O programa fez exatamente o que foi mandado, e o resultado está errado. Pior: **o bug é intermitente** — depende de as duas leituras caírem no mesmo microssegundo. Você pode testar dez vezes e não ver.

As duas ferramentas que resolvem

sync.Mutex — a fechadura

```
var trava sync.Mutex

trava.Lock()           // só uma goroutine passa por aqui de cada vez
if assentos > 0 {
    assentos--
}
trava.Unlock()         // libera a próxima
```

“Mutex” vem de *mutual exclusion*. É o `synchronized` do Java e o `threading.Lock` do Python — conceito idêntico, então você já tem meio caminho andado. Entre o `Lock` e o `Unlock`, **uma goroutine por vez**; as outras esperam na porta.

go test -race — o detector

```
go test -race ./...
```

Este é o trunfo de escolher Go para este PBL. O comando roda seus testes **vigiando todos os acessos à memória**. Se duas goroutines tocarem o mesmo dado sem proteção, ele para e informa: as duas linhas envolvidas, e as duas goroutines culpadas.

POR QUE ISSO VALE NOTA

Nem Python nem Java têm equivalente direto. Você não precisa *argumentar* que tratou concorrência: roda o comando com o bug e ele acusa; coloca o mutex, roda de novo e ele fica em silêncio. É **evidência objetiva**, e é o tipo de coisa que se demonstra ao vivo na apresentação.

10. Como rodar e três experimentos

Rodando

Abra dois terminais na pasta `Aprendizado/01-eco`.

```
Terminal 1: go run ./servidor
Terminal 2: go run ./cliente
```

Digite qualquer coisa no cliente e tecle Enter. A resposta volta com `ECO:` na frente, e o servidor registra o que recebeu.

Comando	O que faz
<code>go run ./servidor</code>	Compila e roda de uma vez, sem gerar arquivo. Ideal para desenvolver.
<code>go build ./servidor</code>	Gera o executável de verdade. É o que vai para o Docker.
<code>go vet ./...</code>	Procura erros comuns que compilam mas estão errados.
<code>gofmt -w .</code>	Formata o código no padrão oficial. Sem debate sobre estilo.

Experimento 1 — ver a concorrência funcionando

Abra **três** terminais de cliente ao mesmo tempo e converse nos três. Todos são atendidos em paralelo, e o servidor mostra três endereços diferentes conectados.

Experimento 2 — ver a concorrência faltando

No servidor, **apague a palavra** `go`, deixando só `atender(conexao)`. Rode de novo com dois clientes.

O segundo cliente conecta, mas **trava**: não recebe resposta nenhuma até o primeiro fechar. Motivo: sem o `go`, a execução entra em `atender` e só volta ao `Accept()` quando aquele cliente sair.

Ver o programa quebrar por falta de uma palavra de duas letras fixa o conceito melhor do que qualquer explicação. Não esqueça de devolver o `go` depois.

Experimento 3 — ver o EOF

Feche um cliente com `Ctrl+C` e observe o servidor imprimir `cliente desconectou`. Confirma na prática o que a seção 6 explicou: **desconexão chega como erro de leitura**, e mata apenas aquela goroutine.

11. Cola de tradução

Para consultar enquanto a sintaxe não fica automática.

Você quer	Python	Java	Go
Criar variável	<code>x = 10</code>	<code>int x = 10;</code>	<code>x := 10</code>
Imprimir	<code>print(x)</code>	<code>System.out.println(x)</code>	<code>fmt.Println(x)</code>
Imprimir formatado	<code>print(f"{a}: {b}")</code>	<code>System.out.printf</code>	<code>fmt.Printf("%v: %v", a, b)</code>
Laço infinito	<code>while True:</code>	<code>while (true) {</code>	<code>for {</code>
Laço contado	<code>for i in range(10):</code>	<code>for (int i=0; i<10; i++)</code>	<code>for i := 0; i < 10; i++</code> <code>{</code>
Percorrer coleção	<code>for x in lista:</code>	<code>for (T x : lista)</code>	<code>for _, x := range lista</code> <code>{</code>
Lista	<code>lista = []</code>	<code>ArrayList<T></code>	<code>var lista []T</code> (slice)
Dicionário	<code>d = {}</code>	<code>HashMap<K,V></code>	<code>m := map[K]V{}</code>
Objeto de dados	<code>class</code> / <code>dataclass</code>	<code>class</code>	<code>type X struct {</code>
Nulo	<code>None</code>	<code>null</code>	<code>nil</code>
Tratar falha	<code>try / except</code>	<code>try / catch</code>	<code>if err != nil {</code>
Garantir limpeza	<code>with</code> / <code>finally</code>	<code>finally</code>	<code>defer</code>
Rodar em paralelo	<code>Thread(...).start()</code>	<code>new</code> <code>Thread(...).start()</code>	<code>go f()</code>
Proteger dado	<code>threading.Lock</code>	<code>synchronized</code>	<code>sync.Mutex</code>
Ignorar retorno	<code>_, y = f()</code>	—	<code>_, y := f()</code>
Servidor TCP	<code>socket.listen()</code>	<code>ServerSocket</code>	<code>net.Listen("tcp", ...)</code>
Cliente TCP	<code>socket.connect()</code>	<code>new Socket(...)</code>	<code>net.Dial("tcp", ...)</code>
JSON	<code>import json</code>	Jackson / Gson	<code>encoding/json</code>

Vocabulário de redes

Termo	Significado
Socket	A tomada por onde um programa fala com a rede.
Porta	Número de 1 a 65535 que identifica <i>qual programa</i> na máquina. O IP acha o computador; a porta acha o processo.
Ouvinte	Socket que só espera conexões. Não transporta dados; produz conexões.
Conexão	Cano privado entre dois programas. Um por cliente.
Bloquear	Parar a execução esperando algo, sem gastar CPU. <code>Accept</code> e <code>ReadString</code> bloqueiam.
EOF	<i>End of file</i> . Em socket, significa que o outro lado fechou.
localhost / ::1	Esta mesma máquina, em IPv4 e em IPv6.
Goroutine	Linha de execução independente, criada com <code>go</code> . Custa ~2 KB.
Condição de corrida	Duas execuções mexendo no mesmo dado ao mesmo tempo, com resultado imprevisível.
Protocolo de aplicação	A regra combinada sobre o formato das mensagens. O nosso: uma linha por mensagem.

12. Próximos passos rumo ao Vaijunto

O exemplo de eco já cobre, funcionando, quatro dos cinco requisitos técnicos do enunciado:

Requisito do PBL	Onde aparece	Situação
Subsistema de rede TCP/IP	<code>net.Listen</code> , <code>net.Dial</code>	Feito
Suporte a API socket básica	<code>Listen</code> , <code>Accept</code> , <code>Dial</code> , ler, escrever	Feito
Concorrência	<code>go atender(conexao)</code>	Feito
Protocolo de aplicação	mensagens delimitadas por <code>\n</code>	Feito
Testado por Docker	—	Depois
Persistência em JSON	—	Depois

O que falta, em ordem de dificuldade

1. Mensagens estruturadas — `encoding/json`

Texto puro não carrega `{"acao": "reservar", "trechos": [1, 2]}`. O pacote converte `struct` em texto e de volta, com duas funções. É a etapa mais fácil da lista.

2. Estado compartilhado — `sync.Mutex`

Onde as goroutines deixam de ser ilhas e passam a disputar a lista de assentos. **É aqui que está a nota do trabalho.** Também é onde `go test -race` deixa de ser curiosidade e vira sua principal ferramenta.

3. Tempo — `time.AfterFunc`

A reserva que expira em 10 minutos se não for paga. Agenda uma função para rodar no futuro — e ela roda **em sua própria goroutine**, o que significa que precisa do mesmo mutex do item anterior. Fácil de esquecer, fácil de errar.

4. Atomicidade multi-trecho

A parte conceitualmente mais difícil: reservar Feira → Salvador → Ilhéus ou **tudo**, ou **nada**. Se o segundo trecho falhar, o primeiro precisa ser desfeito — e isso enquanto outros clientes disputam os mesmos assentos.

5. Persistência em JSON

Salvar em arquivo. Cuidado escondido: gravar em disco **enquanto** goroutines alteram o estado é *outro* problema de concorrência, não apenas de arquivo.

6. Docker

Empacotar. Aqui o Go brilha: binário estático, imagem final de 10 a 20 MB, sem runtime instalado. E é a hora em que aquele detalhe do `":8080"` sem `localhost` decide se funciona ou não.

O RECADO PRINCIPAL DESTE GUIA

Você não vai jogar fora nada do que leu aqui. O `Listen`, o `Accept`, a goroutine por cliente, o `bufio` lendo linha a linha, o `defer` fechando conexão: **tudo isso continua igual no projeto final**. O que muda é apenas o que acontece *dentro* da função `atender`.

O esqueleto está pronto. O resto é preencher.